Canvas AI for Teachers — Quick Reference

Chrome extension + web dashboard + backend, one shared Supabase + pgvector store. Hackathon build, LA Hacks 2026.

01 What's in each workspace

extension/

Chrome extension (Vite + CRXJS + React)

- Injects AI bubble panels into Canvas pages — assignment editor, SpeedGrader, gradebook, course home, module view.
- Uses the teacher's Canvas session cookie (credentials: 'include'), no PAT in the extension at runtime.
- Tap-only UX. The only typed screen is Options — paste Canvas URL + PAT once.
- Talks to backend via JWT; streams BubbleNodes over a chrome.runtime Port.

web/

Next.js 15 dashboard

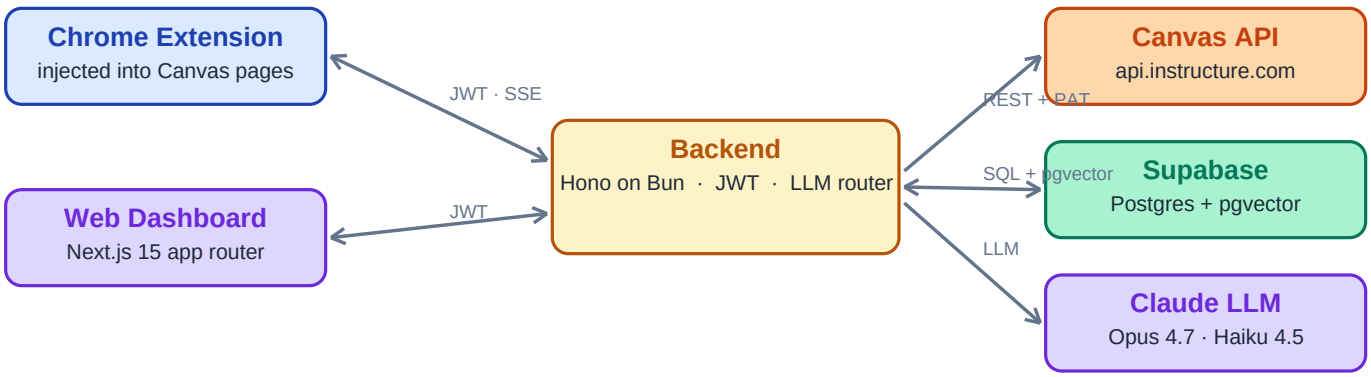
- Landing page, /connect form, /dashboard with multi-course overview.
- /connect is the only typed page — forwards PAT to backend, sets http-only JWT cookie.
- Server Components by default; React Query + Supabase SSR for data.
- For depth workflows: cross-course analytics, scheduled rebuilds, admin.

backend/

Hono on Bun

- Auth: verifies PAT, KMS-wraps it, mints JWT. Never gives PAT back.
- Canvas proxy with per-URL semaphore + 403 backoff.
- LLM router: Opus for authoring, Haiku for scale tasks.
- Retrieval: pgvector top-k cosine over course_embeddings.
- Actions: SSE streams BubbleNode → next bubble → terminal preview.

02 How they fit together



Extension can also call Canvas REST directly from the page (credentials: 'include') — used for writes that must carry the teacher's session, e.g. Push-to-Canvas.

03 How to run (three terminals)

```
# one-time
cp .env.example .env    # fill SUPABASE_*, ANTHROPIC_API_KEY, JWT_SECRET

# Terminal 1 — backend (http://localhost:8787)
cd backend && bun install && bun run dev

# Terminal 2 — web (http://localhost:3000)
cd web && pnpm install && pnpm dev

# Terminal 3 — extension (builds into extension/dist/)
cd extension && pnpm install && pnpm dev
# then: chrome://extensions → Developer mode → Load unpacked → extension/dist
```

04 One happy-path flow (Draft an assignment)

1) Teacher taps "■ AI draft" the extension injected next to Save. 2) Bubble tree asks unit → type → level → length (each row pre-computed by the LLM from class context). 3) Each tap → chrome.runtime.sendMessage → backend /actions/:id/step → pgvector retrieves top-k relevant chunks → Claude returns the next BubbleNode, streamed back over SSE. 4) On ■ Generate, Opus writes the full assignment. Teacher sees a preview diff. 5) Tap "Push to Canvas" → extension writes via Canvas REST with the teacher's session cookie. ═ Other… is the only fallback to typing after onboarding — reserved for the ~5% edge case nothing else covers.